# Workshop cheat sheet

What to do on your laptop, step by step, for steps 01 to 07. Slide numbers are on the right. Every prompt here was tested with fresh agents.

**Before you go on stage**

- Your app is on **On this computer** (Settings → Where rooms live) for steps 01 to 06. The shared hub is only for the 07 recording.
- An empty folder for the live build: `~/workshop-live`. Claude Code signed in, and its folder trust already answered once.
- The deck running: `uv run workshop/serve.py` in the slides repo. The 07 video is ready to play, and the recorded room is open in a browser tab as the backup.
- Tell the room they can watch instead of building (slide 23 says so too).

## 01 · Download and open

slides 25 to 27 · 5 min

1. **Download (25):** show the site's install section. The agent prompt is the easiest path for anyone with a coding agent:

   ```
   Use curl to read https://mycelium-io.github.io/mycelium/agents.md and follow its steps to install Mycelium
   ```

2. **Open (26):** open your app live, and go to a fresh room. Say hello in the room; nobody answers yet. Tell them: the first screen asks for an LLM key. If you have one, you can set it up there. If not, choose **Skip for now**. Nothing today needs one.
3. **The room (27):** click once per callout: rooms, views, tasks, channel, members, memory.

**If stuck:** someone's install fails, so pair them with a neighbor and move on. They can watch.

## 02 · Start a team

slides 28 and 29 · 10 min

1. **Add member** → Your machine → Claude Code → folder `~/workshop-live` → name `planner` → instructions:

   ```
   You turn what the person wants into small tasks on the board and give each to the right teammate. You don't write code. Don't install anything: use only what's already on this machine.
   ```

2. Again, named `builder`:

   ```
   You build what's asked, in this folder. Don't install anything: use only what's already on this machine.
   ```

3. In the room:

   ```
   @planner I want a single HTML page, no build step, that tracks the books I'm reading. Plan it and get @builder started.
   ```

4. **Point at:** tasks appearing on the board within a minute, all for the builder, and the builder picking up the first one in its thread.
5. **It's your own agent (29):** click `builder` in Members to show its live terminal, then **Open its terminal**: it's plain Claude Code. Type nothing that interrupts it.

**If stuck:** an agent sits at a question in its terminal (trust, sign-in), so answer it there. No tasks after a couple of minutes? Post `@!planner` and the line again.

## 03 · Add a reviewer

slide 30 · 5 min

1. Add member → `reviewer`, same folder:

   ```
   You review each change before its task is done. Don't pass anything you haven't tried. Don't install anything: use only what's already on this machine.
   ```

2. In the room:

   ```
   @reviewer you're on the team now: review each task before it's resolved.
   ```

3. **Point at:** the reviewer stays quiet until the builder asks it, by itself, to review its next task. Then it passes it or sends it back.

Mention the wake system here: a mention wakes one agent; `@~name` names one without waking it; `@!name` cuts in.

## 04 · Add a designer

slide 31 · 10 min

1. Add member → `designer`, same folder:

   ```
   You decide how it looks and feels for the people using it. Don't install anything: use only what's already on this machine.
   ```

2. In the room:

   ```
   @designer decide how it should look, save it to memory, and get it restyled to match.
   ```

3. **Point at:** open the style guide in **Memory**, then the restyle task: the builder builds to it, and the designer and reviewer check against the same memory. Open `index.html` in a browser to show the page.

## 05 · Add a security auditor

slide 32 · 10 min

1. Add member → `security`, same folder:

   ```
   You look for security problems in what the team builds and file each one as a task. Don't install anything: use only what's already on this machine.
   ```

2. In the room:

   ```
   @security audit what we have and file what you find as tasks for the team.
   ```

3. **Point at:** its findings land on the board as tasks, and the builder picks them up into the work it's already doing. Nobody told them to coordinate.

## 06 · Accord and Concord

slides 33 to 37 · 15 min

1. **33 to 35:** one sentence per click; read the heading. 34 is Accord, 35 is Concord.
2. **Run Accord (36):** type `/engine conductor`, then file the next thing you want, e.g. `/task Let people rate each book`. In its thread:

   ```
   @conductor accord @~builder @~reviewer @~designer: what is this task?
   ```

   Point at: the main chat's *Accord started* / *ended* lines, then the summary at the top of the thread and its open items.
3. **Run Concord (37):** `/task Pick how to store the books`, then in its thread:

   ```
   @conductor concord @~builder @~reviewer @~designer: which should we pick?
   ```

   Point at: *Picked …*, a bar per agent, and who's least satisfied. **Open decision** shows it in memory.

**If stuck:** `@conductor` does nothing, so it wasn't added: `/engine conductor` and send again. Lots of "only one person said this" is fine; point at the "said in different words" items instead.

## 07 · Demo: one room, two people

slide 38 · demo

1. Play the recorded video of you and Jodee's agents planning a release on the shared hub (how it was recorded: last page).
2. Narrate over it: your `eng` agent and Jodee's `product` agent; Accord flags what doesn't line up; Concord picks, with each one's satisfaction.
3. If someone asks whether it's real, switch to the browser tab with the room.

## Close

slides 39 and 40

The room remembers; your agents stay yours. Leave **Questions** up. If there's time, show the live room: the team has kept building in the background.

# The 07 recording (do this Monday, with Jodee)

Real agents, one each, on mycelium.outshift.com. It took 10 to 12 minutes in the dry runs.

1. **Point the app at the shared hub:** Settings → *Where rooms live* → **On my team's hub** → `https://mycelium.outshift.com`. Sign in if asked, and run `mycelium login` in a terminal so the hub lists your laptop.
2. **Make the room** `release-planning`, click **Invite** and send Jodee the link (**Open in the Mycelium app**).
3. **Add your agent** `eng` (Your machine → Claude Code → empty folder `~/demo`), and click **Start** on the dialog:

   ```
   You speak for engineering: what's built, what's risky and what it costs. Nothing ships half done. Don't install anything or change any files: this is a planning task.
   ```

4. **Jodee adds** `product` on her laptop:

   ```
   You speak for product: what users asked for, the date we announced (November 10), and what the announcement needs. Don't install anything or change any files: this is a planning task.
   ```

5. `/engine conductor`, then `/task Plan the next release` with this description:

   ```
   Our team task app ships one release before the November 10 announcement. Candidates:
   1. Calendar sync: tasks with due dates show up in Google and Outlook calendars
   2. Offline mode: use the app on a plane, sync when back online
   3. Two-factor login
   4. CSV export of any task list
   5. Dark mode
   6. Home-screen widgets on iOS and Android
   There's room for about three of these.
   ```

6. Start recording, then in the task's thread, one at a time, waiting for each to finish:

   ```
   @eng @product what's your position on this release?
   ```

   ```
   @conductor accord @~eng @~product: what is this release?
   ```

   ```
   @conductor concord @~eng @~product: which should we pick?
   ```

7. **Afterwards:** leave the room up (it's the backup), and switch the app back to **On this computer**.

| | |
|---|---|
| **Add to room just waits** | Every start from the shared hub asks first: look for **Start** / **Decline** on that laptop. |
| **Your laptop isn't listed** | Run `mycelium login`; make sure no `mycelium runner` is running in a terminal. |
| **An agent never answers** | Open its terminal from Members and answer any question there, then `@!eng` or `@!product` and the message again. |